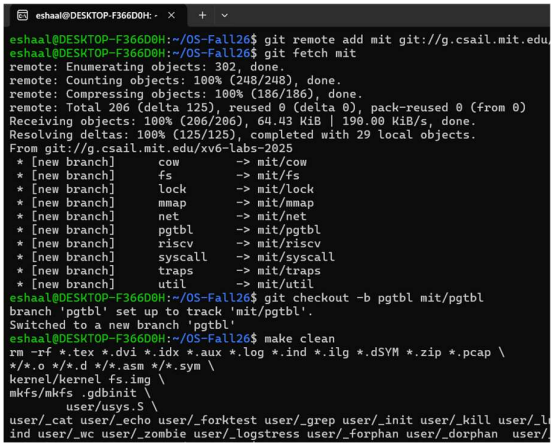
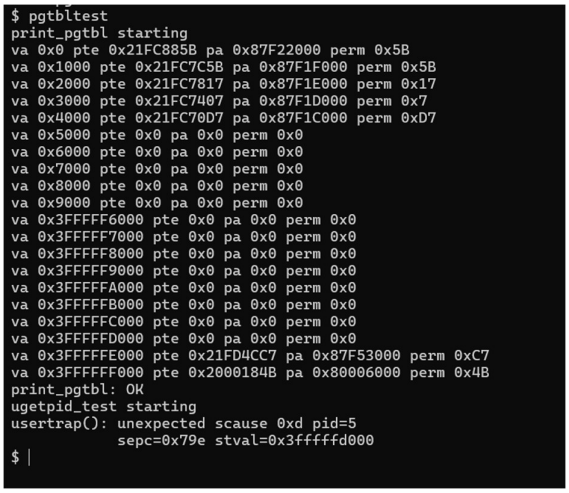
Q1. Inspect a user-process page table

First we connect to mit and bring in the branch



After that we run the pagetable test using pgtbltest on xv6 qemu



va 0x0, 0x1000 perm 0x5B (V,R,X,U,A): text/code. User read+execute, not writable.

va 0x2000 perm 0x17 (V,R,W,U): data/bss. User read/write, not executable; not yet accessed.

va 0x3000 perm 0x07 (V,R,W): guard page. No U bit, so user access faults; catches stack overflow.

va 0x4000 perm 0xD7 (V,R,W,U,A,D): user stack. Read/write, accessed and dirty.

va 0x5000-0x9000 pte 0x0: not valid, unmapped (heap not allocated yet).

va 0x3FFFFF6000-0x3FFFFFD000 pte 0x0: not valid, unmapped.

va 0x3FFFFFE000 perm 0xC7 (V,R,W,A,D): trapframe. Kernel-only (no U); holds saved user registers on traps.

va 0x3FFFFFF000 perm 0x4B (V,R,X,A): trampoline. Kernel-only (no U); code for switching user<->kernel on traps.

Q2(from the 2026 one): Walk and print a two-level page table

eshaal@DESKTOP-F366D0H: ~

GNU nano 7.2 kernel/vm.c *

```
static void
vmprint_level(pagetable_t pagetable, int level, uint64 vabase)
{
  for(int i = 0; i < 512; i++){
    pte_t pte = pagetable[i];
    if((pte & PTE_V) == 0)
      continue;

    uint64 va = vabase + ((uint64)i << PXSHIFT(level));

    for(int d = 0; d < 3 - level; d++)
      printf(" ..");
    printf("%p: pte %p pa %p ", (void*)va, (void*)pte, (void*)PTE2PA(pte));

    if(pte & PTE_R) printf("R");
    if(pte & PTE_W) printf("W");
    if(pte & PTE_X) printf("X");
    if(pte & PTE_U) printf("U");
    printf("\n");

    // not a leaf: recurse into the next-level table
    if((pte & (PTE_R|PTE_W|PTE_X)) == 0)
      vmprint_level((pagetable_t)PTE2PA(pte), level - 1, va);
  }
}

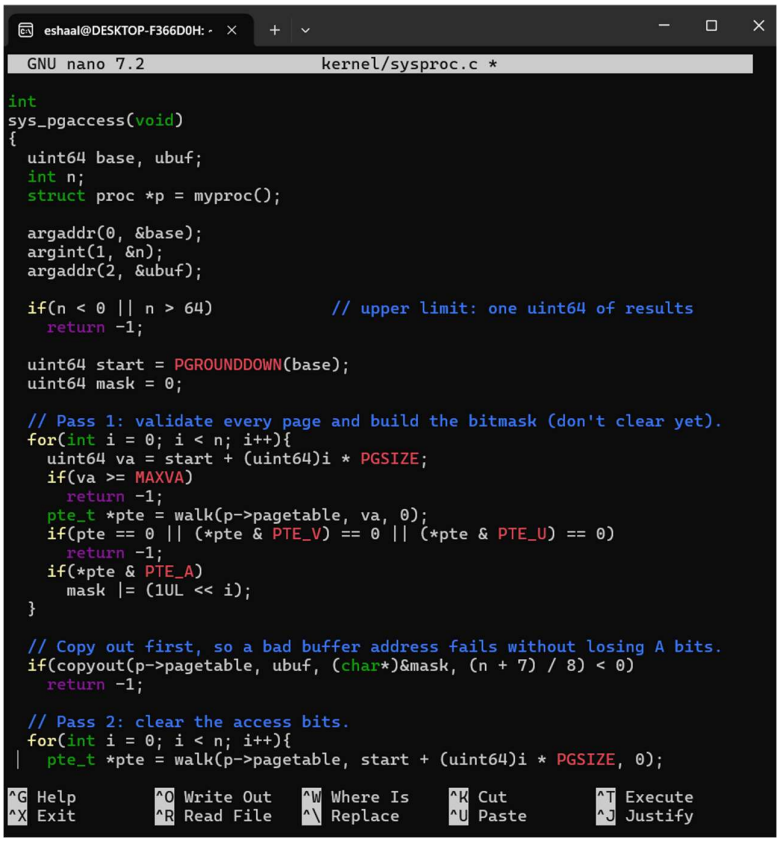

#if defined(LAB_PGTBL) || defined(SOL_MMAP) || defined(SOL_COW)
void
vmprint(pagetable_t pagetable) {
  printf("page table %p\n", pagetable);
  vmprint_level(pagetable, 2, 0);
}
#endif
```

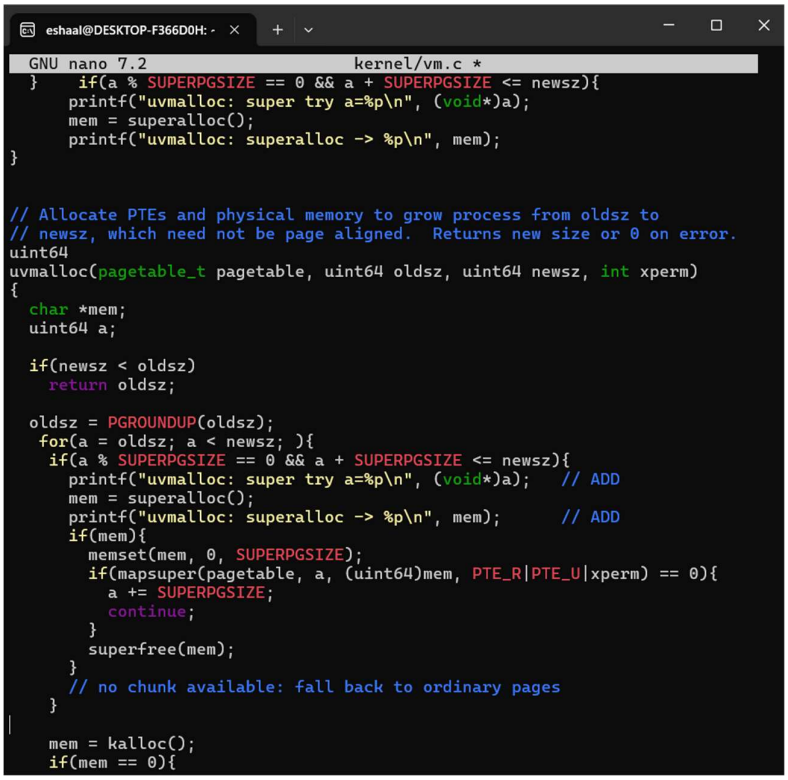
eshaal@DESKTOP-F366D0H: ~

```
hart 2 starting
init: starting sh
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x217C7C5B pa 0x85F1F000 perm 0x5B
va 0x1000 pte 0x217C705B pa 0x85F1C000 perm 0x5B
va 0x2000 pte 0x217C6C17 pa 0x85F1B000 perm 0x17
va 0x3000 pte 0x217C6807 pa 0x85F1A000 perm 0x7
va 0x4000 pte 0x217C64D7 pa 0x85F19000 perm 0xD7
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFD000 pte 0x217D4813 pa 0x85F52000 perm 0x13
va 0x3FFFFFE000 pte 0x217D00C7 pa 0x85F40000 perm 0xC7
va 0x3FFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_pgtbl: OK
ugetpid_test starting
ugetpid_test: OK
print_kpgtbl starting
page table 0x0000000085f22000
 ..0x0000000000000000: pte 0x00000000217c7801 pa 0x0000000085f1e000
 .. ..0x0000000000000000: pte 0x00000000217c7401 pa 0x0000000085f1d000
 .. .. ..0x0000000000000000: pte 0x00000000217c7c5b pa 0x0000000085f1f000 RX
U
 .. .. ..0x0000000000001000: pte 0x00000000217c705b pa 0x0000000085f1c000 RX
U
 .. .. ..0x0000000000002000: pte 0x00000000217c6cd7 pa 0x0000000085f1b000 RW
U
 .. .. ..0x0000000000003000: pte 0x00000000217c6807 pa 0x0000000085f1a000 RW
 .. .. ..0x0000000000004000: pte 0x00000000217c64d7 pa 0x0000000085f19000 RW
U
 ..0x0000003fc0000000: pte 0x00000000217c8401 pa 0x0000000085f21000
```

va	perm	bits	what it contains
0x0	0x5B	V R X U A	**Text (code), page 1.** User can read and execute, not write. A is set because instructions were fetched from it.
0x1000	0x5B	V R X U A	**Text, page 2.** Same as above.
0x2000	0x17	V R W U	**Data/bss.** User read/write, not executable. A and D are clear because it hadn't been touched yet at this point.
0x3000	0x07	V R W	**Guard page.** Mapped but no U bit, so any user access faults. This catches stack overflow.
0x4000	0xD7	V R W U A D	**User stack.** Read/write, accessed and dirty because it is in use.
0x5000–0x9000	0x0	none	**Unmapped.** V=0, so any access page-faults. The heap hasn't been grown yet. The PPN is irrelevant, so pa prints 0.
0x3FFFFF6000–0x3FFFFFC000	0x0	none	**Unmapped** gap at the top of the address space.
0x3FFFFFD000	0x13	V R U	**usyscall page.** Holds struct usyscall (the PID). User read-only: no W, no X. A is clear because it hasn't been read yet.
0x3FFFFFE000	0xC7	V R W A D	**Trapframe.** Saves user registers on a trap. No U bit, so only the kernel can access it.
0x3FFFFFF000	0x4B	V R X A	**Trampoline.** Trap entry/exit code. Executable and readable, not writable, no U bit. Its pa 0x80006000 is inside the kernel image because every process maps the same physical page.

Q3. Detect which pages have been accessed





```
hart 2 starting
hart 1 starting
init: starting sh
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x217C7C5B pa 0x85F1F000 perm 0x5B
va 0x1000 pte 0x217C705B pa 0x85F1C000 perm 0x5B
va 0x2000 pte 0x217C6C17 pa 0x85F1B000 perm 0x17
va 0x3000 pte 0x217C6807 pa 0x85F1A000 perm 0x7
va 0x4000 pte 0x217C64D7 pa 0x85F19000 perm 0xD7
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFD000 pte 0x217D4813 pa 0x85F52000 perm 0x13
va 0x3FFFFFE000 pte 0x217D00C7 pa 0x85F40000 perm 0xC7
va 0x3FFFFFF000 pte 0x20001C4B pa 0x80007000 perm 0x4B
print_pgtbl: OK
ugetpid_test starting
ugetpid_test: OK
print_kpgtbl starting
page table 0x0000000085f22000
 ..0x0000000000000000: pte 0x00000000217c7801 pa 0x0000000085f1e000
 .. ..0x0000000000000000: pte 0x00000000217c7401 pa 0x0000000085f1d000
 .. .. ..0x0000000000000000: pte 0x00000000217c7c5b pa 0x0000000085f1f000 RXU
 .. .. ..0x0000000000001000: pte 0x00000000217c705b pa 0x0000000085f1c000 RXU
 .. .. ..0x0000000000002000: pte 0x00000000217c6cd7 pa 0x0000000085f1b000 RWU
 .. .. ..0x0000000000003000: pte 0x00000000217c6807 pa 0x0000000085f1a000 RW
 .. .. ..0x0000000000004000: pte 0x00000000217c64d7 pa 0x0000000085f19000 RWU
 ..0x0000003fc0000000: pte 0x00000000217c8401 pa 0x0000000085f21000
 .. ..0x0000003fffe00000: pte 0x00000000217c8001 pa 0x0000000085f20000
 .. .. ..0x0000003fffffd000: pte 0x00000000217d4853 pa 0x0000000085f52000 RU
 .. .. ..0x0000003fffffe000: pte 0x00000000217d00c7 pa 0x0000000085f40000 RW
 .. .. ..0x0000003ffffff000: pte 0x0000000020001c4b pa 0x0000000080007000 RX
print_kpgtbl: OK
superpg_fork starting
uvmalloc: super try a=0x0000000000200000
uvmalloc: superalloc -> 0x0000000087e00000
uvmalloc: super try a=0x0000000000400000
uvmalloc: superalloc -> 0x0000000087c00000
uvmalloc: super try a=0x0000000000600000
uvmalloc: superalloc -> 0x0000000087a00000
uvmalloc: super try a=0x0000000000800000
uvmalloc: superalloc -> 0x0000000087800000
uvmalloc: super try a=0x0000000000a00000
uvmalloc: superalloc -> 0x0000000087600000
uvmalloc: super try a=0x0000000000c00000
uvmalloc: superalloc -> 0x0000000087400000
uvmalloc: super try a=0x0000000000e00000
uvmalloc: superalloc -> 0x0000000087200000
usertrap(): unexpected scause 0xf pid=69
            sepc=0x2ec stval=0x5001
superpg_fork: OK
superpg_free starting
uvmalloc: super try a=0x0000000000200000
uvmalloc: superalloc -> 0x0000000087200000
uvmalloc: super try a=0x0000000000400000
uvmalloc: superalloc -> 0x0000000087400000
uvmalloc: super try a=0x0000000000600000
uvmalloc: superalloc -> 0x0000000087600000
uvmalloc: super try a=0x0000000000800000
uvmalloc: superalloc -> 0x0000000087800000
uvmalloc: super try a=0x0000000000a00000
uvmalloc: superalloc -> 0x0000000087a00000
uvmalloc: super try a=0x0000000000c00000
uvmalloc: superalloc -> 0x0000000087c00000
uvmalloc: super try a=0x0000000000e00000
uvmalloc: superalloc -> 0x0000000087e00000
usertrap(): unexpected scause 0xd pid=70
            sepc=0x40a stval=0xfff001
superpg_free: OK
pgtbltest: all tests succeeded
$
```

Q4. Speed up the getpid system call

```
 GNU nano 7.2                        kernel/vm.c *
  if((*pte & PTE_U) == 0)
    return 0;
  pa = PTE2PA(*pte);
  return pa;
}

static void
vmprint_level(pagetable_t pagetable, int level, uint64 vabase)
{
  for(int i = 0; i < 512; i++){
    pte_t pte = pagetable[i];
    if((pte & PTE_V) == 0)
      continue;

    uint64 va = vabase + ((uint64)i << PXSHIFT(level));

    // one " .." per depth: level 2 -> 1, level 1 -> 2, level 0 -> 3
    for(int d = 0; d < 3 - level; d++)
      printf(" ..");
    printf("%p: pte %p pa %p\n", (void*)va, (void*)pte, (void*)PTE2PA(pte));

    // not a leaf (no R/W/X bits): recurse into the next-level table
    if((pte & (PTE_R|PTE_W|PTE_X)) == 0)
      vmprint_level((pagetable_t)PTE2PA(pte), level - 1, va);
  }
}


#if defined(LAB_PGTBL) || defined(SOL_MMAP) || defined(SOL_COW)
void
vmprint(pagetable_t pagetable) {
  printf("page table %p\n", pagetable);
  vmprint_level(pagetable, 2, 0);
}
#endif
```

```
print_pgtbl starting
va 0x0 pte 0x21FC7C5B pa 0x87F1F000 perm 0x5B
va 0x1000 pte 0x21FC705B pa 0x87F1C000 perm 0x5B
va 0x2000 pte 0x21FC6C17 pa 0x87F1B000 perm 0x17
va 0x3000 pte 0x21FC6807 pa 0x87F1A000 perm 0x7
va 0x4000 pte 0x21FC64D7 pa 0x87F19000 perm 0xD7
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFD000 pte 0x21FD4813 pa 0x87F52000 perm 0x13
va 0x3FFFFFE000 pte 0x21FD00C7 pa 0x87F40000 perm 0xC7
va 0x3FFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_pgtbl: OK
ugetpid_test starting
ugetpid_test: OK
print_kpgtbl starting
page table 0x0000000087f22000
 ..0x0000000000000000: pte 0x0000000021fc7801 pa 0x0000000087f1e000
 .. ..0x0000000000000000: pte 0x0000000021fc7401 pa 0x0000000087f1d000
 .. .. ..0x0000000000000000: pte 0x0000000021fc7c5b pa 0x0000000087f1f000
 .. .. ..0x0000000000001000: pte 0x0000000021fc705b pa 0x0000000087f1c000
 .. .. ..0x0000000000002000: pte 0x0000000021fc6cd7 pa 0x0000000087f1b000
 .. .. ..0x0000000000003000: pte 0x0000000021fc6807 pa 0x0000000087f1a000
 .. .. ..0x0000000000004000: pte 0x0000000021fc64d7 pa 0x0000000087f19000
 ..0x0000003fc0000000: pte 0x0000000021fc8401 pa 0x0000000087f21000
 .. ..0x0000003fffe00000: pte 0x0000000021fc8001 pa 0x0000000087f20000
 .. .. ..0x0000003fffffd000: pte 0x0000000021fd4853 pa 0x0000000087f52000
 .. .. ..0x0000003fffffe000: pte 0x0000000021fd00c7 pa 0x0000000087f40000
 .. .. ..0x0000003fffffF000: pte 0x000000002000184b pa 0x0000000080006000
print_kpgtbl: OK
superpg_fork starting
pgtbltest: superpg_fork failed: pte different, pid=3
```

The tree has 3 levels, 2 non-leaf entries at the top, and exactly 8 leaves (7 at the bottom of the low branch… precisely: 5 low leaves and 3 high leaves).

Tree structure

depth	va	perm	meaning
top (level 2)	0x0	0x01	points to a level-1 table covering 0x0–0x3FFFFFFF
top (level 2)	0x3FC0000000	0x01	points to a level-1 table for the top of the address space (index 255)
level 1	0x0	0x01	points to the level-0 table for the low pages
level 1	0x3FFFE00000	0x01	points to the level-0 table for the top pages (index 511)

These four entries have only V set, so they are page-table pages, not data.

The 8 leaf pages

va	pte low bits	bits	what it contains
0x0	0x5B	V R X U A	Text: User read/execute, not writable. A set because instructions were fetched.
0x1000	0x5B	V R X U A	Text: second code page.
0x2000	0xD7	V R W U A D	Data/bss: User read/write, not executable. Now accessed.
0x3000	0x07	V R W	Guard page: No U bit, so user access faults; catches stack overflow.
0x4000	0xD7	V R W U A D	User stack: Writable, accessed, dirty.
0x3FFFFFD000	0x53	V R U A	usyscall page: User read-only (no W, no X). A is set because ugetpid_test read it.
0x3FFFFFE000	0xC7	V R W A D	Trapframe: No U bit, so kernel-only; holds saved user registers on traps.
0x3FFFFFF000	0x4B	V R X A	Trampoline: Executable, no U, no W; kernel-only trap entry/exit code. Its pa 0x80006000 is in the kernel image, since every process maps the same page.

Differences from print_pgtbl:

- **0x2000**: 0x17 earlier, 0xD7 now. The data page was written to between the two prints, so A and D got set.
- **0x3FFFFFD000**: 0x13 earlier, 0x53 now. ugetpid_test read the page in between, so A got set. This confirms the page is being used from user space.
- Everything else (PPNs and permissions) is identical, and the physical addresses are the same pages as before (e.g. 0x87F1F000 for text, 0x80006000 for the trampoline).

- The 0x0 entries from print_pgtbl are absent here because vmprint skips invalid PTEs. That also means the heap range is empty.

Q5.

```
  GNU nano 7.2                    kernel/riscv.h *
#define PGSHIFT 12  // bits of offset within a page

#ifdef LAB_PGTBL
#define SUPERPGSIZE (2 * (1 << 20)) // bytes per page
#define SUPERPGSIZE (2*1024*1024) //
#define SUPERPGROUNDUP(sz)  (((sz)+SUPERPGSIZE-1) & ~(SUPERPGSIZE-1))
#define SUPERPGROUNDDOWN(sz) (SUPERPGROUNDUP(sz)-SUPERPGSIZE)
#endif
```

```
  GNU nano 7.2                    kernel/kalloc.c *

  if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
    panic("kfree");

  // Fill with junk to catch dangling refs.
  memset(pa, 1, PGSIZE);

  r = (struct run*)pa;

  acquire(&kmem.lock);
  r->next = kmem.freelist;
  kmem.freelist = r;
  release(&kmem.lock);
}

// Allocate one 4096-byte page of physical memory.
// Returns a pointer that the kernel can use.
// Returns 0 if the memory cannot be allocated.
void *
kalloc(void)
{
  struct run *r;

  acquire(&kmem.lock);
  r = kmem.freelist;
  if(r)
    kmem.freelist = r->next;
  release(&kmem.lock);

  if(r)
    memset((char*)r, 5, PGSIZE); // fill with junk
  return (void*)r;
}

void* superalloc(void);
void  superfree(void*);
```

```
  GNU nano 7.2                    kernel/kalloc.c *
struct run {
  struct run *next;
};

struct {
  struct spinlock lock;
  struct run *freelist;
} kmem;

#define NSUPER 16                                    // 16 x 2MB = 32MB
#define SUPERBASE (PHYSTOP - NSUPER * SUPERPGSIZE)  // 2MB-aligned

struct {
  struct spinlock lock;
  struct run *freelist;
  int cnt[NSUPER];      // free 4K pages seen so far in each demoted chunk
} supermem;
```

```
void
kinit()
{
  initlock(&kmem.lock, "kmem");
  initlock(&supermem.lock, "supermem");
  freerange(end, (void*)SUPERBASE);
  for(uint64 p = SUPERBASE; p < PHYSTOP; p += SUPERPGSIZE)
    superfree((void*)p);
}
```

```
  GNU nano 7.2                    kernel/kalloc.c *
// initializing the allocator; see kinit above.)
void
kfree(void *pa)
{
  struct run *r;

  if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
    panic("kfree");

  // Fill with junk to catch dangling refs.
  memset(pa, 1, PGSIZE);

    if((uint64)pa >= SUPERBASE){
    // Page of a demoted superpage chunk. Don't put it on the ordinary free
    // list; once all 512 pages are back, return the whole chunk.
    int idx = ((uint64)pa - SUPERBASE) / SUPERPGSIZE;
    int full;
    acquire(&supermem.lock);
    full = (++supermem.cnt[idx] == 512);
    if(full)
      supermem.cnt[idx] = 0;
    release(&supermem.lock);
    if(full)
      superfree((void*)(SUPERBASE + (uint64)idx * SUPERPGSIZE));
    return;
  }

  r = (struct run*)pa;

  acquire(&kmem.lock);
  r->next = kmem.freelist;
  kmem.freelist = r;
  release(&kmem.lock);
}
```

eshaal@DESKTOP-F366D0H:

GNU nano 7.2 kernel/kalloc.c *

```
  if(r)
    memset((char*)r, 5, PGSIZE); // fill with junk
  return (void*)r;
}

void* superalloc(void);
void  superfree(void*);

void*
superalloc(void)
{
  struct run *r;

  acquire(&supermem.lock);
  r = supermem.freelist;
  if(r)
    supermem.freelist = r->next;
  release(&supermem.lock);
  return (void*)r;
}

void
superfree(void *pa)
{
  struct run *r;

  if(((uint64)pa % SUPERPGSIZE) != 0 || (uint64)pa < SUPERBASE ||
     (uint64)pa >= PHYSTOP)
    panic("superfree");

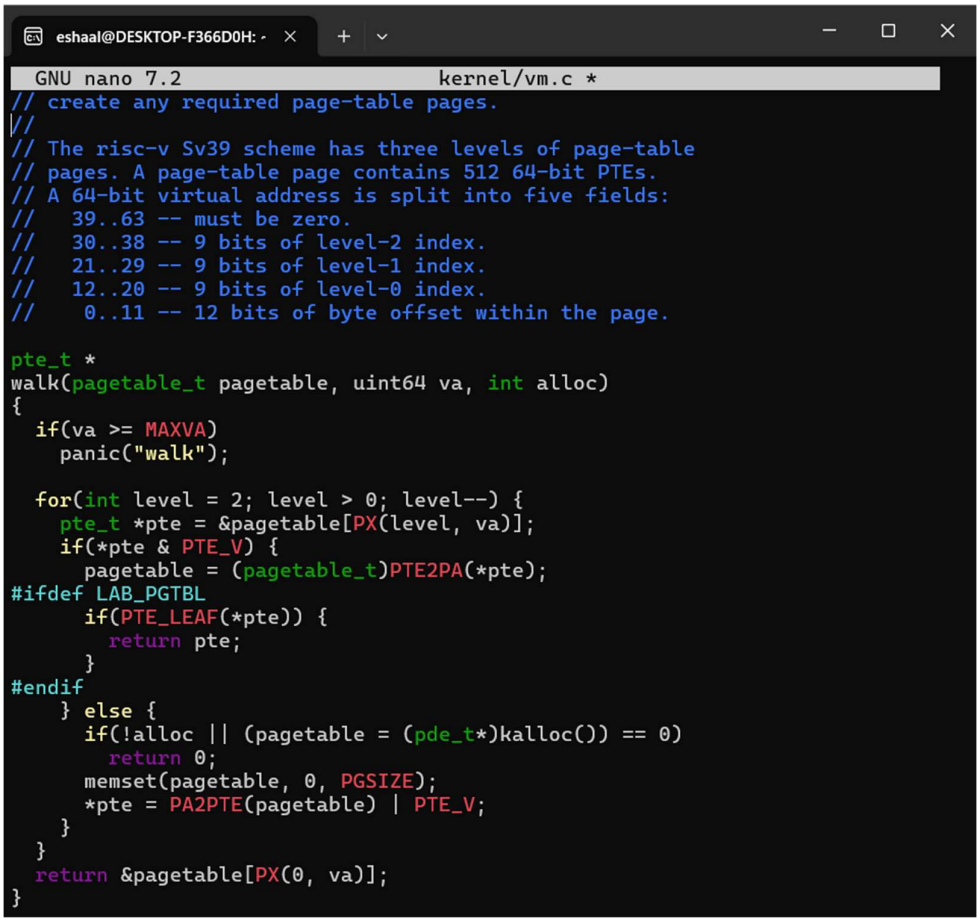
  r = (struct run*)pa;
  acquire(&supermem.lock);
  r->next = supermem.freelist;
  supermem.freelist = r;
  release(&supermem.lock);
}
```

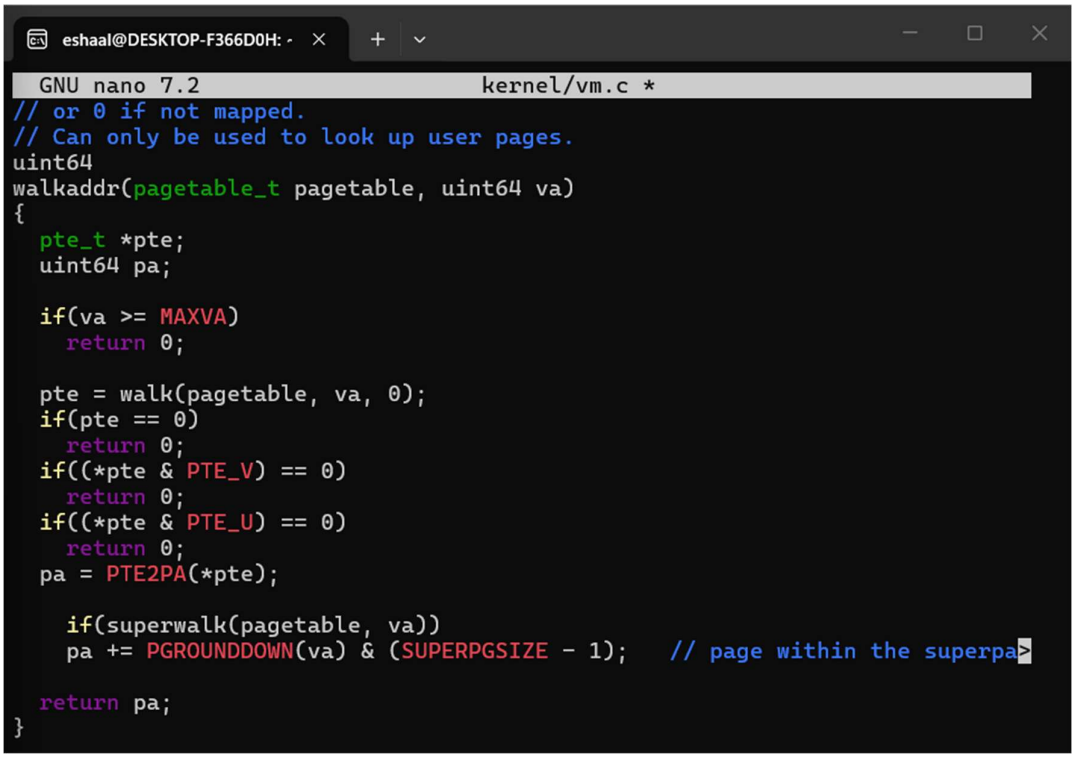
eshaal@DESKTOP-F366D0H:

GNU nano 7.2 kernel/vm.c *

```
// If va lies in a superpage, return its level-1 leaf PTE, else 0.
static pte_t*
superwalk(pagetable_t pagetable, uint64 va)
{
  if(va >= MAXVA)
    return 0;
  pte_t *pte = &pagetable[PX(2, va)];
  if((*pte & PTE_V) == 0)
    return 0;
  pagetable = (pagetable_t)PTE2PA(*pte);
  pte = &pagetable[PX(1, va)];
  if((*pte & PTE_V) && (*pte & (PTE_R|PTE_W|PTE_X)))
    return pte;
  return 0;
}

// Return the address of the PTE in page table pagetable
// that corresponds to virtual address va.  If alloc!=0,
// create any required page-table pages.
//
// The risc-v Sv39 scheme has three levels of page-table
// pages. A page-table page contains 512 64-bit PTEs.
// A 64-bit virtual address is split into five fields:
//   39..63 -- must be zero.
//   30..38 -- 9 bits of level-2 index.
//   21..29 -- 9 bits of level-1 index.
//   12..20 -- 9 bits of level-0 index.
//    0..11 -- 12 bits of byte offset within the page.
```


eshaal@DESKTOP-F366D0H:
GNU nano 7.2 kernel/vm.c *
// create any required page-table pages.
//
// The risc-v Sv39 scheme has three levels of page-table
// pages. A page-table page contains 512 64-bit PTEs.
// A 64-bit virtual address is split into five fields:
//   39..63 -- must be zero.
//   30..38 -- 9 bits of level-2 index.
//   21..29 -- 9 bits of level-1 index.
//   12..20 -- 9 bits of level-0 index.
//   0..11 -- 12 bits of byte offset within the page.
pte_t *
walk(pagetable_t pagetable, uint64 va, int alloc)
{
if(va >= MAXVA)
panic("walk");
for(int level = 2; level > 0; level--) {
pte_t *pte = &pagetable[PX(level, va)];
if(*pte & PTE_V) {
pagetable = (pagetable_t)PTE2PA(*pte);
#ifdef LAB_PGTBL
if(PTE_LEAF(*pte)) {
return pte;
}
#endif
} else {
if(!alloc || (pagetable = (pde_t*)kalloc()) == 0)
return 0;
memset(pagetable, 0, PGSIZE);
*pte = PA2PTE(pagetable) | PTE_V;
}
}
return &pagetable[PX(0, va)];
}


eshaal@DESKTOP-F366D0H:
GNU nano 7.2 kernel/vm.c *
// or 0 if not mapped.
// Can only be used to look up user pages.
uint64
walkaddr(pagetable_t pagetable, uint64 va)
{
pte_t *pte;
uint64 pa;
if(va >= MAXVA)
return 0;
pte = walk(pagetable, va, 0);
if(pte == 0)
return 0;
if((*pte & PTE_V) == 0)
return 0;
if((*pte & PTE_U) == 0)
return 0;
pa = PTE2PA(*pte);
if(superwalk(pagetable, va))
pa += PGROUNDDOWN(va) & (SUPERPGSIZE - 1);   // page within the superpa
return pa;
}

eshaal@DESKTOP-F366D0H:

GNU nano 7.2 kernel/vm.c *

```
// Map a 2MB superpage at va (2MB-aligned) to pa (2MB-aligned).
static int
mapsuper(pagetable_t pagetable, uint64 va, uint64 pa, int perm)
{
  pte_t *pte = &pagetable[PX(2, va)];

  if(*pte & PTE_V){
    pagetable = (pagetable_t)PTE2PA(*pte);
  } else {
    if((pagetable = (pagetable_t)kalloc()) == 0)
      return -1;
    memset(pagetable, 0, PGSIZE);
    *pte = PA2PTE(pagetable) | PTE_V;
  }

  pte = &pagetable[PX(1, va)];
  if(*pte & PTE_V){
    if(*pte & (PTE_R|PTE_W|PTE_X))
      panic("mapsuper: remap");
    // an old level-0 table is left over from earlier sbrk; reuse the slot
    // only if it is completely empty
    pagetable_t l0 = (pagetable_t)PTE2PA(*pte);
    for(int i = 0; i < 512; i++)
      if(l0[i] & PTE_V)
        return -1;
    kfree(l0);
    *pte = 0;
  }
  *pte = PA2PTE(pa) | perm | PTE_V;
  return 0;
}
```

eshaal@DESKTOP-F366D0H:

GNU nano 7.2 kernel/vm.c *

```
// Turn a superpage leaf into a level-0 table of 512 ordinary pages.
static void
demote(pte_t *spte)
{
  uint64 pa = PTE2PA(*spte);
  int flags = PTE_FLAGS(*spte);
  pagetable_t l0 = (pagetable_t)kalloc();

  if(l0 == 0)
    panic("demote: out of memory");
  for(int i = 0; i < 512; i++)
    l0[i] = PA2PTE(pa + (uint64)i * PGSIZE) | flags;
  *spte = PA2PTE(l0) | PTE_V;
}
```

eshaal@DESKTOP-F366D0H:

GNU nano 7.2 kernel/vm.c *

```
void
uvmunmap(pagetable_t pagetable, uint64 va, uint64 npages, int do_free)
{
  uint64 a;
  pte_t *pte;
  int sz = PGSIZE;

  if((va % PGSIZE) != 0)
    panic("uvmunmap: not aligned");

  end = va + npages*PGSIZE;
  for(a = va; a < end; a += sz){
    sz = PGSIZE;
    if((pte = walk(pagetable, a, 0)) == 0) // leaf page table entry allocat>
      continue;
    if((*pte & PTE_V) == 0)  // has physical page been allocated?
      continue;
    if(PTE_FLAGS(*pte) == PTE_V)
      panic("uvmunmap: not a leaf");

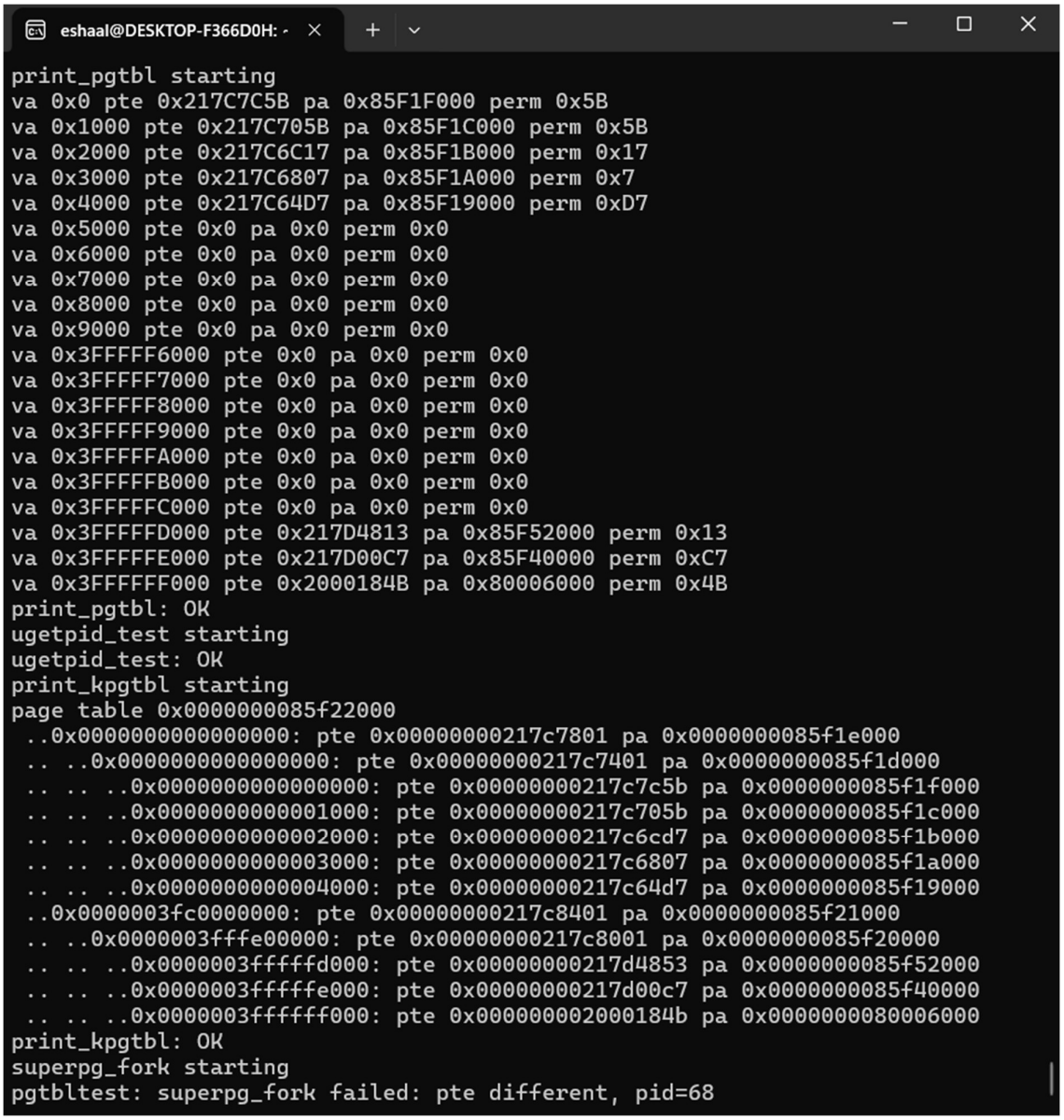
    if(superwalk(pagetable, a) == pte){
      // *pte is a level-1 superpage leaf
      if(a % SUPERPGSIZE == 0 && a + SUPERPGSIZE <= end){
        // the whole superpage is being unmapped
        if(do_free)
          superfree((void*)PTE2PA(*pte));
        *pte = 0;
        sz = SUPERPGSIZE;
        continue;
      }
      // partial unmap: split into 512 ordinary pages, then redo this page
      demote(pte);
      pte = walk(pagetable, a, 0);
    }
```

eshaal@DESKTOP-F366D0H:

GNU nano 7.2 kernel/defs.h *

```
#ifdef LAB_MMAP
gtypedef unsigned long size_t;
typedef long int off_t;
#endif
struct buf;
struct context;
struct file;
struct inode;
struct pipe;
struct proc;
struct spinlock;
struct sleeplock;
struct stat;
struct superblock;
#ifdef LAB_LOCK
struct rwspinlock;
#endif

#ifndef SUPERPGSIZE
#define SUPERPGSIZE (2 * 1024 * 1024)
#endif
```



Two things changed: the physical addresses, and the pid in the last line

Physical addresses

Every ordinary page moved from 0x87F... to 0x85F...

The PTE values changed in the same way (0x21FC... became 0x217C...).

This is a direct effect of the kalloc changes. SUPERBASE is PHYSTOP - 32MB, which is 0x86000000. kinit now gives kalloc() only the memory below that, and the top 32MB goes to the 2MB-chunk pool.

Ordinary pages now come from just under 0x86000000.

The trampoline's pa is still 0x80006000, because it lives in the kernel image and was never allocated by kalloc().

The pid

The last line went from pid=3 to pid=68. xv6 hands out PIDs in increasing order, so 68 just means more processes had run in that boot

The goal

Make sbrk() of 2MB or more use one level-1 leaf PTE (a superpage) instead of 512 ordinary pages, so that superpg_fork and superpg_free pass. Superpages also have to survive fork and partial frees.

The steps taken:

1. Reserved physical memory for 2MB chunks (kalloc.c, defs.h)

- Defined SUPERPGSIZE (2MB) in defs.h, along with the superalloc/superfree prototypes.
- In kalloc.c, set SUPERBASE = PHYSTOP - 32MB. That top 32MB is split into 16 chunks on their own free list (supermem), and kinit gives ordinary kalloc() only the memory below it. Hence the physical addresses shift
- Added superalloc()/superfree() to take and return chunks.
- Added a check in kfree: pages above SUPERBASE belong to a demoted chunk. Once all 512 are freed, the chunk goes back to the pool.

2. Made the page-table code understand level-1 leaves (vm.c)

- walk: the lab's PTE_LEAF check already stops at a superpage
- superwalk: tells whether an address sits in a superpage
- mapsuper: installs a 2MB leaf at level 1
- demote: splits a superpage into a level-0 table of 512 ordinary PTEs
- walkaddr: adds the offset within the superpage

3. Hooked them into the lifecycle (vm.c)

- uvmalloc (reached via growproc and sbrk): if the address is 2MB-aligned and at least 2MB remains, it uses a superpage. Otherwise it uses ordinary pages.

- uvmunmap: frees a whole superpage in one step. For a partial free, it demotes first and then frees the normal pages.
- uvmcopy: when fork copies a superpage, it allocates a new one and copies all 2MB.

2025 LAB Q ACCIDENTALLY DONE:

Q2: Adding syscall to the struct proc

```
 GNU nano 7.2                    kernel/proc.h *
struct proc {
  struct spinlock lock;
  // p->lock must be held when using these:
  enum procstate state;        // Process state
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  int xstate;                  // Exit status to be returned to parent's wa>
  int pid;                     // Process ID

  // wait_lock must be held when using this:
  struct proc *parent;         // Parent process

  // these are private to the process, so p->lock need not be held.
  uint64 kstack;               // Virtual address of kernel stack
  uint64 sz;                   // Size of process memory (bytes)
  pagetable_t pagetable;       // User page table
  struct trapframe *trapframe; // data page for trampoline.S
  struct context context;      // swtch() here to run process
  struct file *ofile[NOFILE];  // Open files
  struct inode *cwd;           // Current directory
  char name[16];               // Process name (debugging)

  struct usyscall *usyscall //shared read-only page
  pgtbl_t pagetable //user pagetable
};
```

```
 GNU nano 7.2                    kernel/proc.c *
  return 0;

found:
  p->pid = allocpid();
  p->state = USED;

  // Allocate a trapframe page.
  if((p->trapframe = (struct trapframe *)kalloc()) == 0){
    freeproc(p);
    release(&p->lock);
    return 0;
  }

  // allocating a usyscall page
  if((p->usyscall = (struct usyscall *) kalloc())==0){
    freeproc(p);
    release(&p->lock);
    return(0);
  }
  p->usyscall->pid=p->pid;

  // An empty user page table.
  p->pagetable = proc_pagetable(p);
  if(p->pagetable == 0){
    freeproc(p);
    release(&p->lock);
    return 0;
  }
```

```
 GNU nano 7.2                        kernel/proc.c *
  // map the trapframe page just below the trampoline page, for
  // trampoline.S.
  if(mappages(pagetable, TRAPFRAME, PGSIZE,
              (uint64)(p->trapframe), PTE_R | PTE_W) < 0){
    uvmunmap(pagetable, TRAMPOLINE, 1, 0);
    uvmfree(pagetable, 0);
    return 0;
  }

  //mapping the shared usyscall page, it will be read-only for user
  if(mappages(pagetable, USYSCALL, PGSIZE, (uint64)(p->usyscall), PTE_R |PT>
    uvmunmap(pagetable, TRAPFRAME, 1, 0);
    uvmunmap(pagetable, TRAMPOLINE, 1, 0);
    uvmfree(pagetable,0);
    return 0;
   }
  return pagetable;
}

// Free a process's page table, and free the
// physical memory it refers to.
void
proc_freepagetable(pagetable_t pagetable, uint64 sz)
{
  uvmunmap(pagetable, TRAMPOLINE, 1, 0);
  uvmunmap(pagetable, TRAPFRAME, 1, 0);
  uvmunmap(pagetable, USYSCALL, 1, 0); //adding an unmapping tofree usyscall
  uvmfree(pagetable, sz);
}
```

```
 GNU nano 7.2                        kernel/proc.c *
// p->lock must be held.
static void
freeproc(struct proc *p)
{
  if(p->trapframe)
    kfree((void*)p->trapframe);
  p->trapframe = 0;
  if(p->pagetable)
    proc_freepagetable(p->pagetable, p->sz);

  //added to free the physical page and clear the pointer
  if(p->usyscall)
    kfree((void*)p->usyscall);
  p->usyscall = 0;
```

After all the changes:

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x21FC7C5B pa 0x87F1F000 perm 0x5B
va 0x1000 pte 0x21FC705B pa 0x87F1C000 perm 0x5B
va 0x2000 pte 0x21FC6C17 pa 0x87F1B000 perm 0x17
va 0x3000 pte 0x21FC6807 pa 0x87F1A000 perm 0x7
va 0x4000 pte 0x21FC64D7 pa 0x87F19000 perm 0xD7
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFD000 pte 0x21FD4813 pa 0x87F52000 perm 0x13
va 0x3FFFFFE000 pte 0x21FD00C7 pa 0x87F40000 perm 0xC7
va 0x3FFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_pgtbl: OK
ugetpid_test starting
ugetpid_test: OK
print_kpgtbl starting
print_kpgtbl: OK
superpg_fork starting
pgtbltest: superpg_fork failed: pte different, pid=3
$
```

Changes:
The main change is at va 0x3FFFFFD000, the page just below the trapframe. Everything else in the printout is the same as before (only the physical addresses shifted, which is normal since kalloc() hands out different pages each boot).

The code changes that caused it:
- kernel/proc.h: added struct usyscall *usyscall; to struct proc.
- allocproc(): kalloc() a page and store p->pid in it.
- proc_pagetable(): mappages() that page at USYSCALL with PTE_R | PTE_U.
- proc_freepagetable(): uvmunmap() the USYSCALL page.
- freeproc(): kfree() the physical page and clear the pointer.

Which other syscalls could use this page?

The pattern works for syscalls that only read a small piece of kernel data and have no side effects.

- **getppid()**: the parent's PID could be stored in the shared page when the process is created. When a parent exits, its children are reparented to init, so the kernel would have to update each child's page at that point.
- **uptime()**: it only returns the kernel's ticks counter. So we can't use a per-process page, because the value changes on every timer interrupt. Instead, the kernel would map one global read-only page into every process and update ticks there in clockintr(). User code would then read it directly.
- **sbrk(0)**: calling it with 0 just returns the current heap size. The kernel could keep p->sz mirrored in the page and update it whenever the size changes (in growproc).

Syscalls that change state or need privileged work can't be done this way: fork, exec, read, write, sleep, kill, open, and so on. They have to trap into the kernel.